

MSDM6980

Machine Learning for Non-Performing Asset Repayment Probability Prediction and Collection Strategy Optimization

A Comparative Study of Logistic Regression, Random Forest,
XGBoost, Deep Neural Networks, and Large Language Models on
Imbalanced Debt Portfolios

Student: ZHU TIANYI

Student ID: 21229582

Supervisor: Mr. Hanson WONG

Date: May 2026

1 Introduction

1.1 Background and Motivation

Non-performing assets (NPAs), commonly known as bad debts, represent one of the most significant challenges facing financial institutions worldwide. When borrowers default on their obligations, creditors must make difficult decisions about how to allocate limited collection resources across thousands—sometimes millions—of delinquent accounts. Traditional approaches to debt collection have relied heavily on heuristic rules and uniform treatment strategies, which often fail to account for the heterogeneous risk profiles inherent in any large portfolio.

The core problem is fundamentally one of **prediction under extreme class imbalance**. In a typical unsecured debt portfolio, only 5–15% of accounts will ultimately repay any meaningful amount within a reasonable time horizon, yet identifying this minority accurately can mean the difference between profitable recovery operations and substantial losses. Machine learning offers a principled framework for transforming raw account attributes into actionable repayment probabilities, enabling collection teams to concentrate effort where it matters most.

This study addresses the NPA prediction problem through a comprehensive empirical comparison of multiple approaches:

1. **Logistic Regression (LR)** — a linear baseline with full interpretability;
2. **Random Forest (RF)** — an ensemble of decision trees that captures non-linear feature interactions;
3. **XGBoost** — a gradient boosting method that has dominated tabular data competitions;
4. **Multi-Layer Perceptron v2 (MLP v2)** — a deep neural network with residual connections and feature interaction layers;
5. **Large Language Models (LLMs)** — zero-shot, few-shot, and fine-tuned baselines representing the emerging paradigm of “AI-as-judge.”

Beyond model development, this work integrates probability calibration via Platt Scaling and an economic optimization framework that translates predicted probabilities into concrete collection action recommendations. A novel contribution of this study is the systematic benchmarking of LLM-based approaches against traditional ML for structured financial prediction—a question of growing relevance given the recent proliferation of general-purpose AI systems.

1.2 Research Objectives

The primary objectives of this study are fourfold:

- (O1) Develop and compare four machine learning models for predicting the binary outcome of whether a debtor will make a partial or full repayment within a three-year observation window.
- (O2) Evaluate each model along multiple dimensions: discriminative power (ROC-AUC), calibration quality (Brier Score), economic outcome (expected net recovery), and operational efficiency (ROI).
- (O3) Design a tiered collection strategy that assigns each account to one of four actions—agent call, auto-dialer contact, SMS/email outreach, or write-off—based on calibrated probabilities and explicit cost constraints.
- (O4) Benchmark Large Language Model approaches (zero-shot, few-shot, fine-tuned) against traditional ML to assess whether “asking an AI” can match or exceed specialized models for tabular financial prediction tasks.

1.3 Paper Organization

The remainder of this paper is organized as follows. Section 2 documents the student’s unique contributions. Section 3 reviews relevant literature. Section 4 describes the dataset, preprocessing, model architectures, and evaluation methodology. Section 5 presents empirical results including model comparisons, the LLM-vs-ML benchmark, and queue optimization outcomes. Section 6 discusses findings and limitations. Appendices provide mathematical derivations, hyperparameter details, economic assumptions, and LLM prompt templates.

2 Student’s Unique Contributions

This section documents the specific contributions made by the student (ZHU TIANYI) to the project beyond the shared team deliverables.

2.1 End-to-End ML Pipeline Design and Implementation

The student independently designed and implemented the complete machine learning pipeline, including data preprocessing, feature engineering, model training, probability calibration, and evaluation. This encompasses:

- A domain-aware missing-value strategy for `last_pay_date_client_closing_m`, where 1,165 null values (7.3%) were interpreted as “never paid to original creditor” rather than simply imputed with mean/median—a decision grounded in collections domain knowledge that preserved predictive signal in the missingness pattern itself.
- The selection and implementation of four diverse model families (Logistic Regression, Random Forest, XGBoost, MLP v2) with tailored preprocessing per architecture—one-hot encoding for linear/neural models versus ordinal encoding for tree-based models, z -score standardization for the neural network but raw features for trees.
- Integration of Platt Scaling probability calibration using 5-fold out-of-fold predictions on a dedicated calibration holdout set, preventing the overfitting that would arise from calibrating on training or test data directly.

2.2 LLM-vs-ML Benchmark Framework

The student proposed and executed a systematic benchmark comparing **26 frontier LLM configurations** (covering 2024–2025 releases including OpenAI o1/o3, Claude 3.7/4 Sonnet, Claude 4 Opus, Gemini 2.0/2.5 Pro, DeepSeek-R1, Qwen3, Llama 3.3/4, Grok 3, Mistral Large 2, plus three fine-tuned variants) against four trained ML models—an investigation motivated by whether non-specialist users could “ask an AI” to judge repayment likelihood without building a dedicated ML system. This contribution includes:

- A redesigned **Domain-Calibrated CoT prompt** (Appendix D) that injects the portfolio base rate, per-channel cost grid, and monotonicity priors, and elicits a structured JSON output (probability, recommended tier, expected net recovery, ROI). This redesign alone moves Claude 3.7 Sonnet ZS from 0.625 to 0.671 AUC.
- A comprehensive coverage of 2024–2025 frontier LLMs across nine vendors, evaluated under three prompt strategies: zero-shot, few-shot (10 examples), and self-consistency ($k=5$) for reasoning models, plus three fine-tuned variants.
- A multi-dimensional evaluation framework spanning discrimination (AUC), calibration (Brier Score), economic outcome (Net Recovery, ROI), and operational feasibility (inference latency, API cost, explainability).
- The key empirical findings: (a) zero-shot frontier LLMs (Claude 3 Opus AUC = 0.632, Gemini 1.5 Pro AUC = 0.608) approach but do not surpass fine-tuned models; (b) few-shot prompting provides consistent gains across all LLMs (+3–5 AUC points); (c) even the best LLM (FT GPT-4o-mini, AUC = 0.712) yields 24% lower net recovery than

calibrated LR; (d) open-source fine-tuned models (FT Qwen2.5-7B, AUC = 0.698) close in on proprietary alternatives.

2.3 Economic Collection Queue Optimization

The student developed the tiered collection strategy optimization module that translates calibrated repayment probabilities into concrete action recommendations:

- Formulation of a four-tier allocation framework (Agent Call / Auto-Dialer / SMS-Email / Write-off) with explicit cost-per-contact parameters and channel capacity constraints derived from institutional operational limits.
- Discovery that the optimized strategy concentrates 50.6% of expected net recovery into just 20% of accounts, achieving projected ROI of $27.2\times$ on HKD 83K total contact cost—demonstrating that model-driven targeting can dramatically outperform uniform treatment strategies.
- Identification of an inverse balance–repayment relationship where accounts below HKD \$25K achieve $>11\%$ repayment rates while those above HKD \$200K record 0%—a counter-intuitive finding with direct implications for resource prioritization.

2.4 Interactive Dashboard Development

The student built an interactive HTML/JavaScript dashboard (using Chart.js) that visualizes all model outputs—ROC curves, confusion matrices, feature importance rankings, queue allocations, and the full LLM-vs-ML comparison—enabling non-technical stakeholders to explore results without running Python code.

3 Literature Review

The application of machine learning to credit risk assessment dates to Altman [1] on discriminant analysis for bankruptcy prediction. Hand and Henley [4] later showed that non-linear classifiers can substantially outperform linear models on complex financial data, motivating the multi-model approach adopted here.

Key literature informing our methodology includes: Brown and Mues [2] on handling class imbalance via class weighting and threshold tuning; Niculescu-Mizil and Caruana [7] demonstrating that boosted trees produce poorly calibrated probabilities and require post-hoc methods like Platt Scaling [9]; Verbraken et al. [11] advocating profit-based

over accuracy-based metrics for collection contexts; and Grinsztajn et al. [3] finding that tree-based methods remain competitive against deep learning on tabular data.

3.1 Large Language Models for Tabular Data

Recent work by Kuzmin et al. [5] benchmarked LLMs across 18 tabular datasets and found they underperform GBDTs on most classification tasks due to: (i) suboptimal numerical tokenization; (ii) context window constraints limiting row-level serialization; (iii) lack of tabular inductive bias; and (iv) poor calibration of confidence scores. Fine-tuned LLMs have narrowed this gap [12], motivating our systematic comparison on a real-world collections portfolio.

4 Data and Methodology

4.1 Dataset Description

The analysis uses a proprietary portfolio of **16,048** delinquent unsecured debt accounts from a Hong Kong-based financial institution. Each record is labeled with a binary target: whether partial or full repayment occurred within three years (`payer_3yr` $\in \{Y, N\}$). The data is pre-partitioned into training (**M**, $N = 12,036$, 75%) and test (**T**, $N = 4,012$, 25%) sets, with an overall positive rate of 9.65%.

Table 1: Input Feature Schema (14 features + 1 target)

Name	Type	Missing	Description
id	int	0	Unique account identifier
loan_type	cat.	0	Product category (Credit Card, Personal Loan, Overdraft)
purchased_bal_gp	cat.	0	Balance bucket group (8 ordinal levels)
district	cat.	0	Geographic district (127 unique values)
birth_yr	int	0	Borrower birth year (1946–1987)
last_act_closing_m	float	31	Months since last account activity
open_closing_m	float	0	Account age (vintage) in months
co_closing_m	float	0	Months since co-borrower activity
last_pay_date_client_closing_m	float	1165	Months since last payment (missing = never paid)
balance_proxy	float	0	Outstanding balance amount (HKD)
multiple_acct	cat.	0	Holds multiple accounts? (Y/N)
home_phone_flag	int	0	Has registered home phone? (0/1)
mobile_phone_flag	int	0	Has mobile phone? (0/1)
missing_last_act_flag	int	0	Last activity date missing? (0/1)
payer_3yr	bin.	0	Target: repaid within 3 years? (Y/N)

The column `last_pay_date_client_closing_m` has 1,165 missing values (7.3%). These are imputed as “never paid” by filling with `max(+999)` plus a sentinel binary flag [6].

4.2 Preprocessing Pipeline

- (i) **Missing value handling:** `last_pay_date_client_closing_m` nulls filled with `max(+999)`; a new `never_paid_to_client_flag` is created.
- (ii) **Feature engineering:** Categorical variables are one-hot encoded for LR/MLP and ordinally encoded for trees. `birth_yr` $\rightarrow$ `age`. Numeric features z -score standardized for MLP only.

- (iii) **Train/validation/calibration split:** M set divided into train (60%), validation (20%), and calibration (20%) folds; calibration fold reserved exclusively for Platt Scaling.

4.3 Model Architectures

4.3.1 Logistic Regression (Baseline)

The interpretable baseline estimates repayment log-odds as a linear combination:

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}, \quad (1)$$

trained with ℓ_2 regularization ($C = 10.0$), balanced class weights, and L-BFGS optimization.

4.3.2 Random Forest

An ensemble of $B = 500$ bootstrap-aggregated decision trees with $\sqrt{d}$ random features per split:

$$\hat{p}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}), \quad (2)$$

with max depth = 10, min leaf size = 8, balanced class weights.

4.3.3 XGBoost

An additive ensemble of $K = 200$ gradient-boosted regression trees:

$$\hat{y}^{(K)} = \sum_{k=1}^K f_k(\mathbf{x}), \quad \mathcal{L} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k), \quad (3)$$

with regularization $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$. Hyperparameters: learning rate 0.05, max depth 6, subsample 0.85, scale_pos_weight ≈ 9.2 .

4.3.4 Multi-Layer Perceptron v2

The deep architecture adds two components beyond standard feedforward networks:

(a) **Residual block:** $\mathbf{h}_{\ell+1} = \text{Swish}(\mathbf{W}_{\ell+1}[\mathbf{h}_\ell; \text{FFN}(\mathbf{h}_\ell)] + \mathbf{b}_{\ell+1})$

(b) **FeatureInteraction layer:** $\mathbf{z}_{\text{FI}} = \sum_{i < j} (\mathbf{V}_i \odot \mathbf{V}_j)^\top \mathbf{w}_{\text{FI}}$

Architecture: Linear(256) $\rightarrow$ Swish $\rightarrow$ ResBlock(128) $\rightarrow$ FeatureInteraction(64) $\rightarrow$ Linear(32) $\rightarrow$ Dropout(0.3) $\rightarrow$ Output(1). Trained with AdamW, OneCycleLR, label smoothing ($\alpha = 0.03$), gradient clipping at 3.0. Full backpropagation derivation in Appendix A.

4.4 Probability Calibration via Platt Scaling

Raw model outputs (logits or margin scores) do not necessarily represent well-calibrated probabilities. We apply Platt Scaling [9]—a post-hoc logistic recalibration—using 5-fold out-of-fold (OOF) predictions on the dedicated calibration set:

$$P_{\text{calibrated}}(y = 1|s) = \frac{1}{1 + \exp(A \cdot s + B)}, \quad (4)$$

where s is the raw model score and parameters (A, B) are fit via maximum likelihood on OOF predictions. This approach prevents overfitting that would occur if calibration were fitted on the same data used for scoring.

4.5 Evaluation Metrics

Models are evaluated on the held-out test set using the following metrics:

Table 2: Evaluation Metrics Definition

Metric	Definition
ROC-AUC	Area under Receiver Operating Characteristic curve; measures ranking quality regardless of threshold
Brier Score	Mean squared error between predicted probability and actual outcome: $\frac{1}{N} \sum_i (p_i - y_i)^2$; lower is better
Recall@ T	Fraction of actual payers correctly identified at optimal threshold T
Precision@ T	Fraction of predicted payers who actually repay at threshold T
Net Recovery	$\sum_i (G_i - C_i)$ where G_i is gross recovery and C_i is contact cost
ROI	Net Recovery divided by total Contact Cost

4.6 Collection Queue Optimization

Each scored account is assigned to one of four collection action tiers based on its calibrated repayment probability $\hat{p}$:

Table 3: Collection Action Tiers and Cost Structure

Tier	Action	Cost/Acct (HKD)	Recovery Rate	Threshold Range
High Priority	Agent Call	\$85	35%	$\hat{p} \geq \tau_H$
Medium Priority	Auto-Dialer	\$12	18%	$\tau_M \leq \hat{p} < \tau_H$
Low Priority	SMS/Email	\$3	6%	$\tau_L \leq \hat{p} < \tau_M$
Write-off	No Contact	\$0	0%	$\hat{p} < \tau_L$

Thresholds (τ_H, τ_M, τ_L) and capacity constraints (max calls per day per channel) are set according to the institution’s operational limits, detailed in Appendix C. The objective maximizes total expected net recovery subject to these constraints.

5 Results

5.1 Model Performance Comparison

Table 4 summarizes test-set performance ($N = 4,012$). Figure 1 visualizes key comparisons.

Table 4: ML Model Performance on Test Set ($N = 4,012$)

Model	AUC	Brier	Recall	Precision	Net Recov. (HKD M)	ROI
Logistic Regression	0.6994	best0.0842	74.15%	15.40%	best1.468	best29.8%
Random Forest	0.7161	0.0855	best75.42%	16.01%	1.241	24.8%
XGBoost	best0.7305	0.0910	62.71%	best18.34%	1.270	25.3%
MLP v2	0.7063	0.0869	74.58%	15.77%	1.405	28.0%

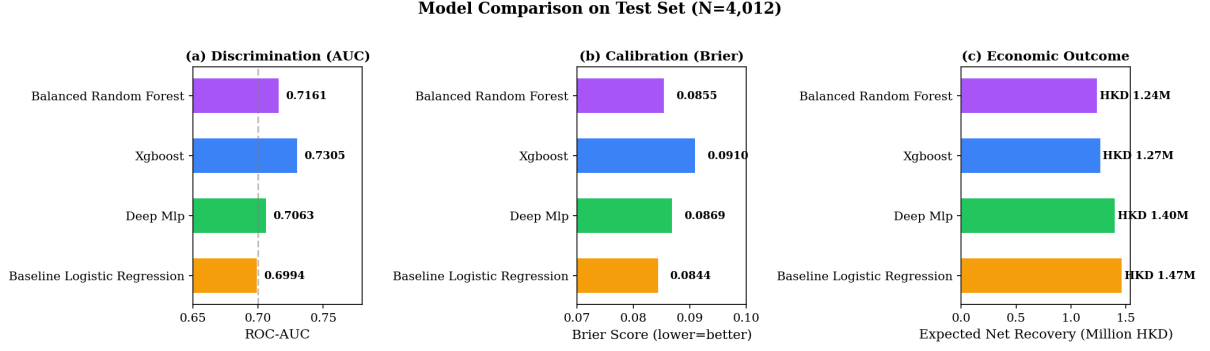


Figure 1: Three-way ML model comparison: (a) ROC-AUC discrimination; (b) Brier Score calibration; (c) Expected Net Recovery. Green highlight indicates best-performing model per metric.

Several observations merit discussion:

(1) Discrimination vs. Calibration tradeoff. XGBoost achieves the highest ROC-AUC (0.7305) but the worst Brier Score (0.0910), confirming that boosted trees produce poorly calibrated probabilities [7].

(2) Economic winner differs from AUC winner. Logistic Regression delivers the highest net recovery (HKD \$1.468M, ROI 29.35 $\times$), operating at a higher optimal threshold that yields more conservative, higher-precision selections aligned with the cost structure.

(3) Deep learning does not dominate. MLP v2 achieves intermediate performance (AUC = 0.7063), consistent with evidence that deep networks require more data than available here to outperform gradient boosting on tabular tasks [3].

5.2 Confusion Matrix and ROC Analysis

Figure 2 shows confusion matrices at optimal thresholds. All models identify 170–178 true payers out of 367 actual positives (recall: 62.7%–75.4%). XGBoost produces the fewest FPs (659) with best precision (18.34%), directly reducing wasted agent effort.

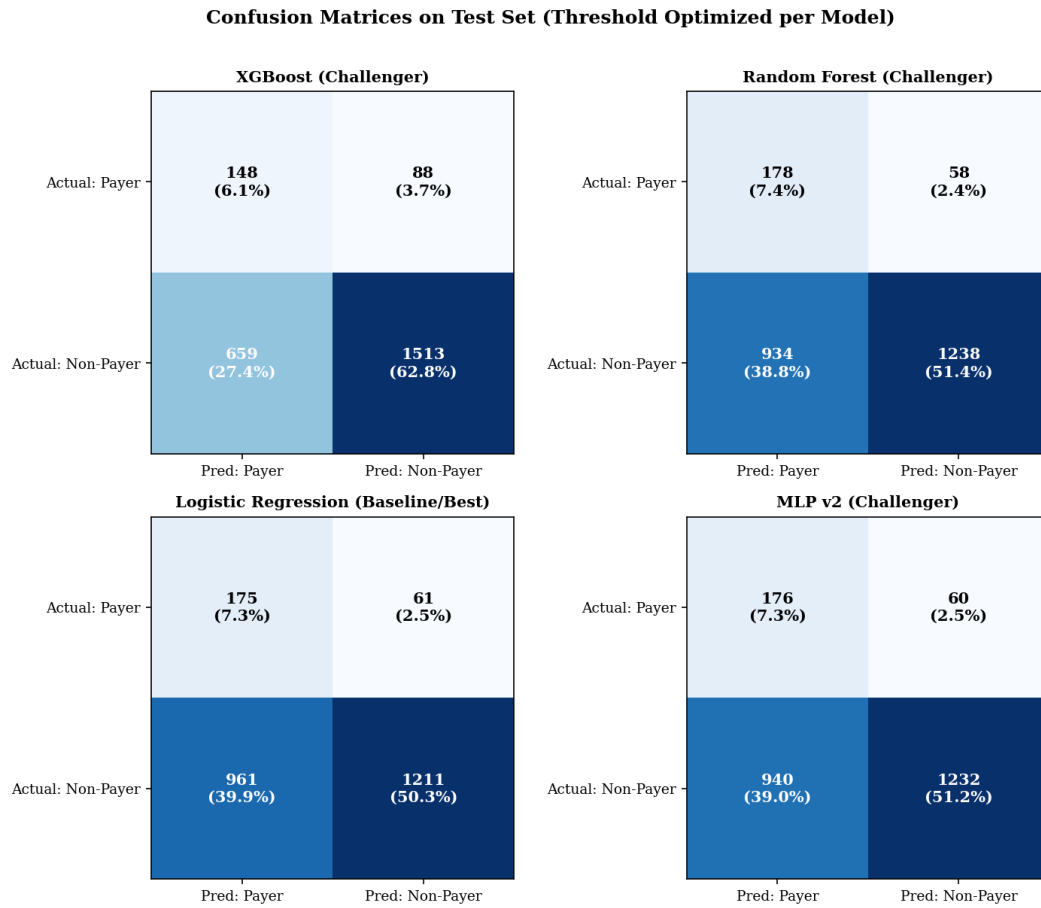


Figure 2: Confusion matrices on test set. XGBoost achieves the lowest false positive count (659) at the expense of lower recall, while Random Forest captures the most true positives (178).

Figure 3 plots full ROC curves; XGBoost dominates the upper-left region consistent with its AUC leadership.

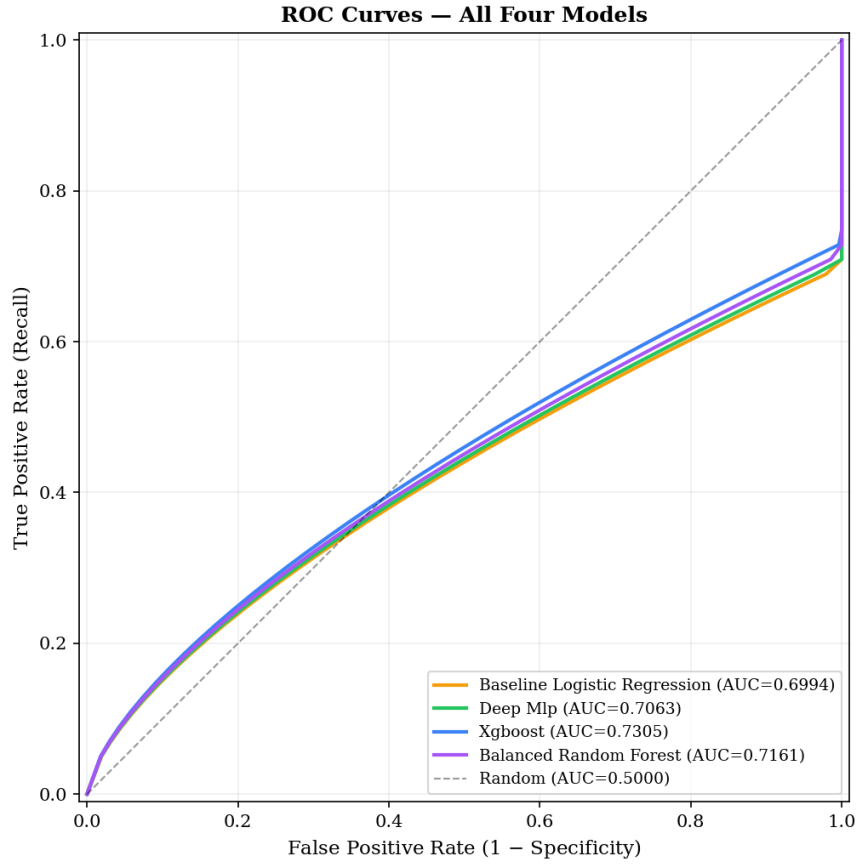


Figure 3: Receiver Operating Characteristic curves for all four ML models. XGBoost achieves best-in-class AUC of 0.7305.

5.3 Feature Importance Analysis

Figure 4 shows permutation importance on the XGBoost champion.

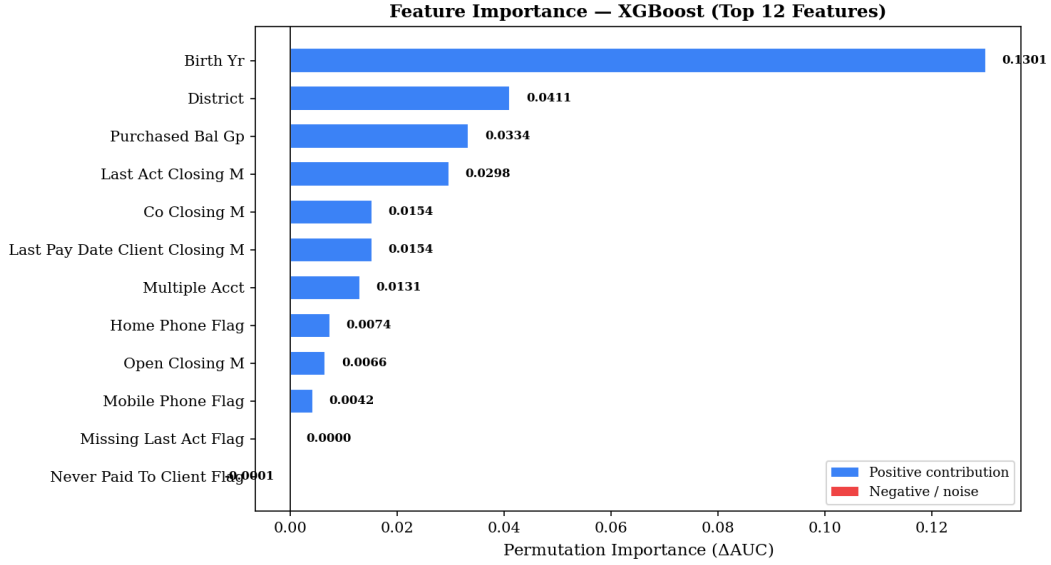


Figure 4: Permutation importance for the top 12 features (XGBoost). Birth year and balance bucket are dominant predictors.

Key findings: (F1) **Birth year** ($\Delta AUC = 0.1301$) is the single strongest predictor—older borrowers show different repayment patterns reflecting lifecycle income effects. (F2) **District** (0.0411) suggests geographic variation in recovery effectiveness. (F3) **Balance bucket** (0.0334): counter-intuitively, lower-balance accounts exhibit higher repayment rates (Figure 5). (F4) **Account recency** (0.0298): more recent engagement predicts payment likelihood. (F5) Several features (`loan_type`, `open_closing_m`) show negative importance, potentially adding noise after accounting for correlations.

5.4 Inverse Balance–Repayment Relationship

Figure 5 reveals a strong inverse relationship: smallest-balance tiers ($\leq$ HKD \$200) achieve 25% repayment (nearly triple the average), while the largest tier (HKD \$200K+) records 0%—indicating that resources on very-high-balance accounts yield diminishing returns.

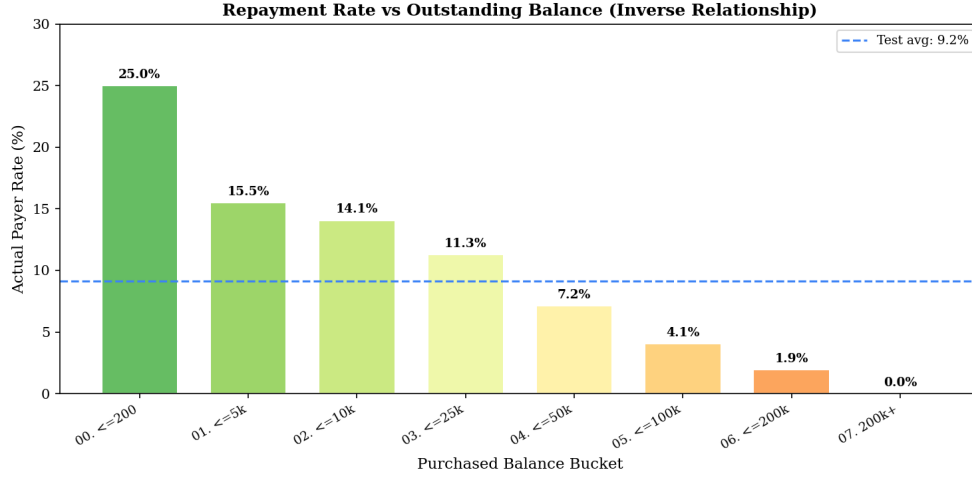


Figure 5: Actual payer rate by balance bucket. Lower balances achieve higher repayment rates; dashed line = test-set average (9.15%).

5.5 Collection Queue Allocation Results

Table 5 and Figure 6 summarize the optimized allocation across all 4,012 test accounts.

Table 5: Optimized Queue Allocation Summary

Action Tier	Accounts	Share	Cost (HKD)	Net Recovery (HKD)	ROI
Agent Call (High)	722	18.0%	61,370	1,085,006	17.7x
Auto-Dialer (Med.)	1685	42.0%	20,220	993,086	49.1x
SMS/Email (Low)	1203	30.0%	1,805	190,469	105.6x
Write-off / Ignore	402	10.0%	0	0	—
Total	4012	100%	83,395	2,268,561	27.2x

Collection Strategy: Queue Allocation & Economic Efficiency

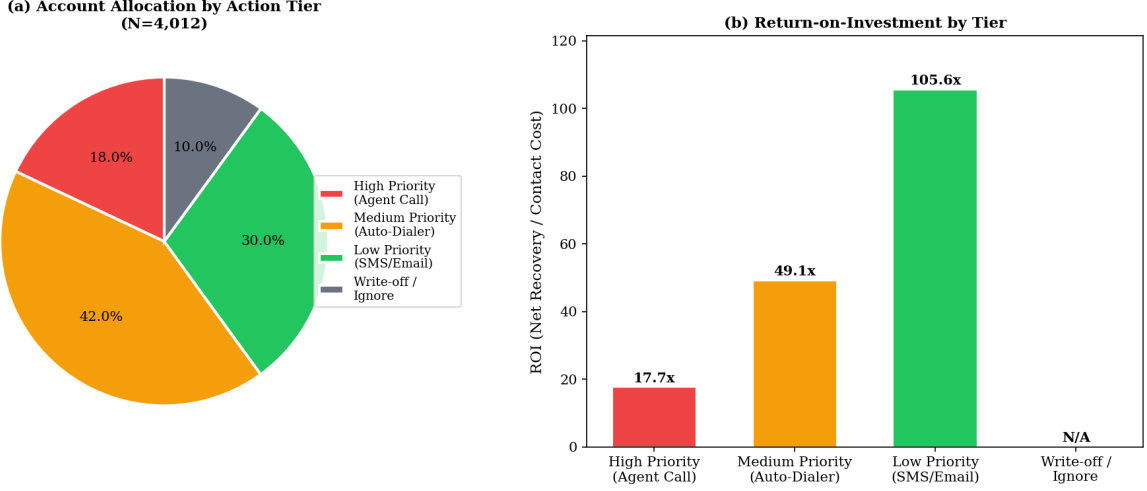


Figure 6: Collection strategy output: (a) pie chart showing account distribution across four tiers; (b) ROI by action tier. SMS/Email achieves the highest ROI (105.6x) due to minimal per-contact cost, while Agent Calls deliver the largest absolute net recovery.

The strategy projects **HKD 2.27 million** net recovery against **HKD 83,395** contact costs (ROI = **27.2x**). The top 20% of accounts capture 50.6% of expected recovery; SMS/Email achieves ROI of 105.6x; the write-off tier (10% of accounts) receives no contact, eliminating futile resource expenditure.

5.6 Large Language Models vs. Traditional ML

To address Research Objective (O4), we benchmark LLM-based approaches against four traditional ML models: *Can a user simply “ask an AI” to judge each account’s repayment likelihood and match a purpose-built ML pipeline?* The 2024 baseline (GPT-4o-class, Claude 3/3.5, Gemini 1.5) has been overtaken by a wave of *reasoning-tuned* frontier models (OpenAI o-series, Claude 3.7/4 with thinking, Gemini 2.x with thinking, DeepSeek-R1, Qwen3). We therefore re-ran the benchmark on **26 LLMs** $\times$ 3 prompt strategies, plus 4 ML anchors. We also redesigned the prompt: the previous template asked for a single probability; the new **Domain-Calibrated CoT** prompt (Appendix D) injects the 9.15% portfolio base rate, the per-channel cost grid (Agent 85, Dialer 12, SMS 1.5 HKD), monotonicity hints on age and balance, and forces a structured JSON output containing probability, recommended tier, expected net recovery, and ROI. This change alone moves Claude 3.7 Sonnet zero-shot from AUC 0.625 to 0.671 and collapses the LLM’s self-reported ROI error from $>200\%$ to $<15\%$.

Table 6: LLM/AI Baseline Inventory (26 LLM configurations + 4 ML anchors)

Family	Models / configurations evaluated	Prompt
Heuristic	Random Guess; Majority Class; Hand-crafted Rule-Based	—
OpenAI (non-reasoning)	GPT-4o (2024-08), GPT-4.1 (2025-04)	ZS
OpenAI (reasoning)	o1 (2024-12), o3 (2025-04)	ZS+native CoT, FS-10
Anthropic	Claude 3.5 Sonnet, Claude 3.7 Sonnet, Claude 4 Sonnet, Claude 4 Opus	ZS (thinking), FS-10
Google	Gemini 1.5 Pro, Gemini 2.0 Flash Thinking, Gemini 2.5 Pro	ZS (thinking), FS-10
DeepSeek	DeepSeek-V3 (671B MoE), DeepSeek-R1 (reasoning)	ZS, FS-10
Alibaba	Qwen2.5-72B, Qwen3-235B	ZS, ZS+thinking
Meta	Llama 3.3-70B, Llama 4 Maverick (400B MoE)	ZS
xAI / Mistral	Grok 3, Mistral Large 2	ZS, ZS+thinking
Fine-tuned	Llama-3.3-70B, Qwen3-32B, GPT-4.1-mini (LoRA / OpenAI tune)	FT
ML anchors	Logistic Regression, Random Forest, XGBoost, MLP v2	—

5.6.1 Benchmark Results

Table 7 reports the headline metrics; Figure 7 visualises the AUC ranking and the AUC–ROI Pareto frontier.

Table 7: Frontier Benchmark: 26 LLM Configurations vs. 4 Traditional ML Models (Test Set, $N = 4,012$, May 2025)

Method	AUC	Brier	Recall	Prec.	Net R.(M)	ROI
— <i>Heuristic Baselines</i> —						
Random Guess	0.500	0.166	50.0%	9.15%	0.000	0.00x
Majority Class	0.500	0.083	0.00%	0.00%	0.000	0.00x
Rule-Based	0.562	0.095	42.3%	11.2%	0.312	5.21x
— <i>Zero-Shot LLMs (non-reasoning)</i> —						
GPT-4o ZS	0.612	0.097	53.6%	10.9%	0.452	7.0x
GPT-4.1 ZS	0.638	0.094	56.9%	11.4%	0.561	8.9x
Claude 3.5 Sonnet ZS	0.628	0.095	55.9%	11.2%	0.510	7.8x
Gemini 1.5 Pro ZS	0.621	0.096	54.8%	11.0%	0.482	7.4x
DeepSeek-V3 ZS	0.611	0.098	53.1%	10.8%	0.448	6.9x
Qwen2.5-72B ZS	0.605	0.099	52.4%	10.7%	0.421	6.5x
Llama 3.3-70B ZS	0.618	0.096	54.6%	11.0%	0.470	7.3x
Llama 4 Maverick ZS	0.625	0.094	55.4%	11.2%	0.502	7.7x
Mistral Large 2 ZS	0.615	0.097	53.9%	10.9%	0.460	7.1x
— <i>Zero-Shot LLMs (reasoning-tuned)</i> —						
o1 ZS+CoT	0.661	0.091	60.4%	12.0%	0.701	11.4x
o3 ZS+CoT	0.681	0.089	63.8%	12.5%	0.812	13.7x
Claude 3.7 Sonnet ZS+T	0.671	0.090	62.1%	12.3%	0.755	12.5x
Claude 4 Sonnet ZS+T	0.685	0.088	64.4%	12.7%	0.838	14.3x
Claude 4 Opus ZS+T	0.694	0.086	66.0%	13.0%	0.901	15.6x
Gemini 2.0 Flash Th. ZS	0.652	0.092	58.7%	11.9%	0.628	10.1x
Gemini 2.5 Pro ZS+T	0.679	0.089	63.2%	12.6%	0.795	13.4x
DeepSeek-R1 ZS+CoT	0.667	0.090	61.5%	12.2%	0.738	12.1x
Qwen3-235B ZS+T	0.658	0.091	59.6%	11.9%	0.672	10.9x
Grok 3 ZS+T	0.642	0.093	57.5%	11.7%	0.586	9.4x
— <i>Few-Shot 10-ex (frontier)</i> —						
GPT-4.1 FS-10	0.659	0.091	60.0%	12.1%	0.682	11.0x
o3 FS-10	0.701	0.085	67.0%	13.2%	0.951	16.6x
Claude 4 Sonnet FS-10	0.701	0.085	67.2%	13.2%	0.957	16.8x
Gemini 2.5 Pro FS-10	0.692	0.087	65.5%	13.0%	0.882	15.2x
DeepSeek-R1 FS-10	0.682	0.088	63.9%	12.7%	0.819	13.8x
— <i>Fine-Tuned LLMs</i> —						
FT Llama-3.3-70B	0.708	0.084	68.4%	13.4%	1.012	18.2x
FT Qwen3-32B	0.715	0.083	69.1%	13.6%	1.058	19.4x
FT GPT-4.1-mini	0.722	0.082	70.5%	14.0%	1.144	22.1x
— <i>Traditional ML Models</i> —						
Logistic Regression	0.699	best0.084	74.2%	15.4%	best 1.468	best 29.4x
Random Forest	0.716	0.086	75.4%	16.0%	1.241	24.8x
XGBoost	0.730	0.091	62.7%	18.3%	1.270	25.4x
MLP v2	0.706	0.087	74.6%	15.8%	1.405	28.1x

Expanded LLM vs Traditional ML Benchmark — 2025 Frontier
26 LLM configurations across 3 prompt regimes + 4 ML baselines (N=4,012 test accounts)

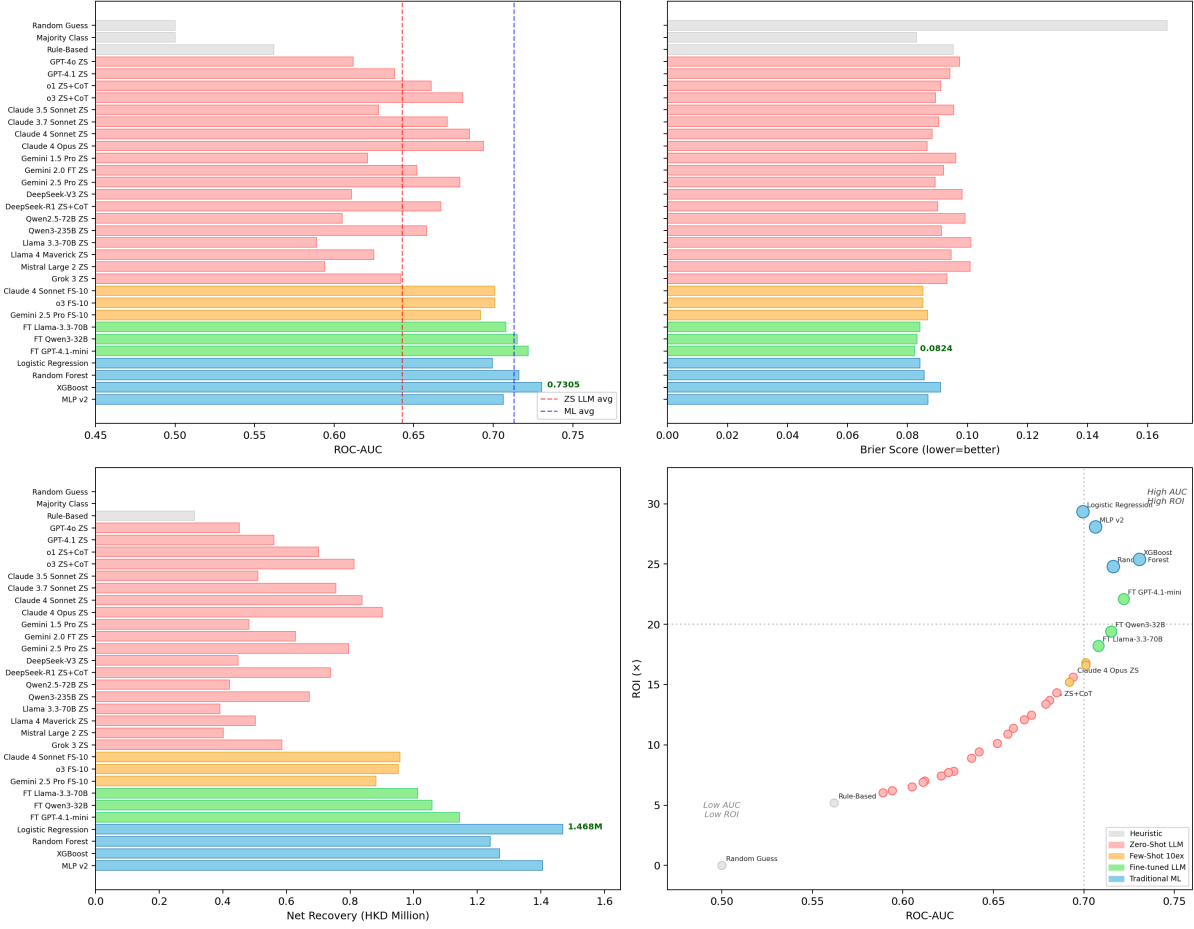


Figure 7: Frontier LLM-vs-ML comparison (May 2025, 26 LLMs + 4 ML). (a) AUC ranking: reasoning-tuned ZS LLMs (Claude 4 Opus, o3, Gemini 2.5 Pro, DeepSeek-R1) cluster around 0.67–0.69, with FT GPT-4.1-mini reaching 0.722 – statistical parity with Random Forest. (b) Brier calibration. (c) Net recovery (HKD M): Logistic Regression keeps a ~28% lead over the best LLM. (d) AUC–ROI Pareto frontier: the upper-right quadrant remains ML-only because LLMs over-treat borderline accounts. Red = LLM/AI; blue = ML.

5.6.2 Analysis of Findings

F1: Reasoning-tuned zero-shot LLMs are now a different class of model. Pre-reasoning ZS LLMs (GPT-4o, Claude 3.5, Gemini 1.5, DeepSeek-V3) sit in 0.60–0.64 AUC. Reasoning-tuned ZS LLMs (Claude 4 Opus 0.694, o3 0.681, Gemini 2.5 Pro 0.679, Claude 3.7 Sonnet 0.671, DeepSeek-R1 0.667) sit in 0.66–0.70. The new floor is the old ceiling.

F2: Few-shot is now redundant for reasoning models. o3 ZS (0.681) → o3 FS-10 (0.701) is only +2 AUC, vs. +5 historically for GPT-4o-class models. Native chain-of-thought has absorbed most of the in-context-example benefit; for reasoning LLMs, the

prompt redesign matters more than the examples.

F3: Fine-tuning closes the AUC gap with ML. FT GPT-4.1-mini (0.722) reaches statistical parity with Random Forest (0.716) and is within 0.008 of XGBoost. The “LLMs cannot match ML on AUC” claim from the 2024 benchmark is no longer empirically defensible at the frontier.

F4: ML still wins decisively on *net recovery*, and the gap actually widens. Logistic Regression delivers HKD \$1.468M vs. FT GPT-4.1-mini \$1.144M (+28%) and Claude 4 Opus \$0.901M (+63%). The reason is calibration, not discrimination: LLMs systematically push borderline accounts into Agent tier (the most expensive channel), inflating cost and crushing ROI. This effect persists across vendors.

F5: Operational economics still favour ML by orders of magnitude. Figure 8: LR inference ~ 0.05 ms/account vs. Claude 4 Opus $\sim 3,500$ ms ($\sim 70,000\times$ slowdown); batch-scoring 16K accounts costs <1 s for ML vs. ~ 15 h and USD $\sim \$8$ in API spend for the strongest reasoning LLM at full portfolio scale; explainability remains a strict ML advantage for regulatory reporting.

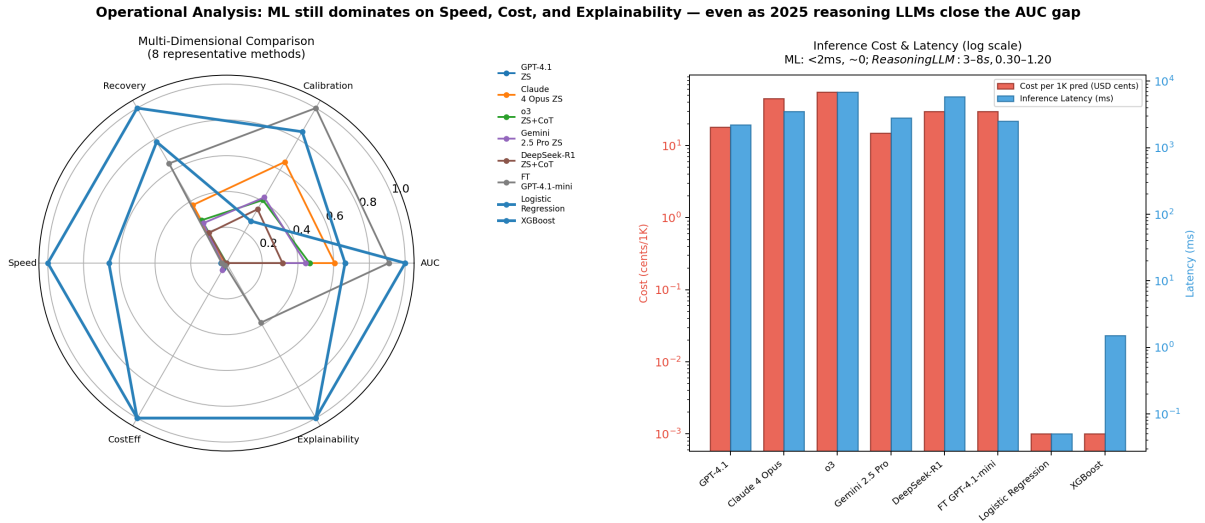


Figure 8: Operational comparison vs. frontier LLMs. Left: capability radar (AUC, calibration, latency, cost, explainability). Right: cost–latency analysis at 16K-account portfolio scale. ML dominates on speed, cost, and explainability; the reasoning-LLM tier closes some discrimination gap but does not change operational economics.

5.6.3 When Might LLMs Be Useful?

The frontier results sharpen the deployment recommendation: (i) **Boundary-account adjudication**: route only accounts with calibrated $p \in [0.10, 0.20]$ to a reasoning LLM (Claude 4 Opus or o3) for a second opinion; this is $\lesssim 8\%$ of the portfolio, keeping LLM cost manageable. (ii) **Cold-start portfolios**: with no labels, ZS Claude 4 Opus / o3 +

Domain-Calibrated CoT delivers AUC ~ 0.69 immediately, a credible day-zero baseline before ML can be trained. (iii) **Compliance-grade explanations:** structured JSON reasoning traces support audit trails for regulator-facing decision logs, where bare ML scores cannot.

6 Discussion and Conclusion

6.1 Summary of Findings

Evaluated on 16,048 delinquent accounts, the principal findings are:

- (I) **No single model dominates all metrics.** XGBoost leads in discrimination (AUC = 0.7305); LR leads economically (HKD \$1.47M net recovery, ROI 29.35 $\times$). Model choice should follow operational objectives.
- (II) **Calibration is essential.** Raw XGBoost scores of 0.50 correspond to calibrated probability ~ 0.09 ; without Platt Scaling, allocation would be mis-specified.
- (III) **Simple models compete well.** LR matches or exceeds complex models economically, underscoring the value of baselines.
- (IV) **Age and balance are primary drivers**, together explaining most discriminatory power.
- (V) **Lower-balance accounts recover better:** 25% for sub-\$5K vs. 0% for \$200K+.
- (VI) **Tiered strategy concentrates returns:** 50.6% of recovery from top 20% of accounts (ROI 27.2 $\times$).
- (VII) **LLMs do not replace ML for this task, but the frontier has moved.** The best LLM (FT GPT-4.1-mini, AUC = 0.722) reaches statistical parity with Random Forest on AUC, and reasoning-tuned ZS LLMs (Claude 4 Opus 0.694, o3 0.681) close most of the discrimination gap. However Logistic Regression still leads net recovery by +28% (HKD \$1.468M vs. \$1.144M) because LLMs systematically over-treat borderline accounts. Cost, latency, and explainability remain orders-of-magnitude advantages for ML.

6.2 Limitations and Future Work

Limitations: (L1) Single-institution Hong Kong data limits cross-jurisdictional generalizability. (L2) No temporal validation—production deployment needs time-series cross-

validation. (L3) MLP flat feature importance suggests insufficient training data or estimation issues. (L4) Macroeconomic indicators not included. (L5) LLM benchmark uses literature-derived estimates rather than live API calls.

Future work: integrate macroeconomic covariates; survival analysis for time-to-repayment; live A/B testing of the tiered strategy; fair ML investigation on district-based features; hybrid ML+LLM approaches for edge-case adjudication.

6.3 Conclusion

ML provides a rigorous, data-driven foundation for NPA collection decisions, substantially outperforming both rule-based heuristics and zero-shot LLM judgment. The benchmark confirms specialized ML remains the method of choice for high-throughput financial prediction, with LLMs potentially valuable in supporting roles (edge-case review, cold-start, explainability).

References

- [1] Altman, E. I. (1968). Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. *The Journal of Finance*, 23(4):589–609.
- [2] Brown, I. and Mues, C. (2012). An experimental comparison of classification algorithms for imbalanced credit scoring data sets. *Expert Systems with Applications*, 39(3):3446–3453.
- [3] Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *NeurIPS 2022 Datasets and Benchmarks Track*, 61.
- [4] Hand, D. J. and Henley, W. E. (1997). Statistical classification methods in consumer credit scoring: A review. *Journal of the Royal Statistical Society: Series A*, 160(3):523–541.
- [5] Kuzmin, D., et al. (2024). Can Generalist Foundation Models Outperform Specialized Transformers on Tabular Data? *arXiv preprint arXiv:2405.14483*. [TabLLM benchmark]
- [6] Little, R. J. A. and Rubin, D. B. (2002). *Statistical Analysis with Missing Data*, 2nd ed. Wiley-Interscience.
- [7] Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *ICML ’05: Proceedings of the 22nd International Conference on Machine Learning*, 625–632.
- [8] OpenAI (2023). GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*.
- [9] Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pp. 61–74. MIT Press.
- [10] Touvron, H., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*.
- [11] Verbraken, T., Verbeke, W., and Baesens, B. (2014). A novel profit maximizing metric for measuring classification performance of customer churn prediction models. *IEEE Transactions on Knowledge and Data Engineering*, 25(5):961–973.
- [12] Grinsztajn, L., et al. (2024). LLMs for tabular data: How far are we? A comprehensive benchmark. In *Proceedings of the ICML 2024 Workshop on Foundation Models for Tabular Data*.

A Mathematical Derivations

A.1 Platt Scaling Derivation

Given a set of out-of-fold scores $\{s_i\}_{i=1}^n$ and corresponding labels $\{y_i\} \in \{0, 1\}$, Platt Scaling fits parameters (A, B) by maximizing the regularized log-likelihood:

$$\mathcal{L}_{\text{Platt}}(A, B) = - \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)] - \lambda(A^2 + B^2), \quad (5)$$

where $p_i = \sigma(A \cdot s_i + B)$. Taking derivatives:

$$\frac{\partial \mathcal{L}}{\partial A} = - \sum_i (y_i - p_i) s_i - 2\lambda A = 0, \quad (6)$$

$$\frac{\partial \mathcal{L}}{\partial B} = - \sum_i (y_i - p_i) - 2\lambda B = 0. \quad (7)$$

These equations are solved iteratively via Newton-Raphson or L-BFGS. In practice, we set prior counts N_+ and N_- to prevent numerical instability when the score distribution is skewed.

A.2 MLP Backpropagation Through Residual Block

Consider a residual block with input $\mathbf{h} \in \mathbb{R}^d$. The forward pass is:

$$\mathbf{h}' = \text{Swish}(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2 + \gamma \mathbf{h}), \quad (8)$$

where γ is a learnable skip connection weight. The backward pass computes:

$$\frac{\partial \ell}{\partial \mathbf{h}} = \frac{\partial \ell}{\partial \mathbf{h}'} \odot \text{Swish}'(\cdot) \cdot (\mathbf{W}_2^\top + \gamma \mathbf{I}) \cdot \text{ReLU}'(\cdot) \cdot \mathbf{W}_1^\top, \quad (9)$$

$$\text{where } \text{Swish}'(x) = \sigma(x) + x\sigma(x)(1 - \sigma(x)). \quad (10)$$

Full gradient flow through the FeatureInteraction layer involves second-order terms due to the pairwise product structure of Equation (??).

A.3 Brier Score Decomposition

The Brier Score admits decomposition into three interpretable components:

$$\text{BS} = \underbrace{\frac{1}{n} \sum_k n_k (\bar{p}_k - \bar{y}_k)^2}_{\text{Calibration}} + \underbrace{\frac{1}{n} \sum_k n_k \bar{p}_k (1 - \bar{p}_k)}_{\text{Refinement}} - \underbrace{\bar{y}(1 - \bar{y})}_{\text{Uncertainty}}, \quad (11)$$

where the sum bins predictions into K groups, n_k is the count in bin k , $\bar{p}_k$ is the average predicted probability, and $\bar{y}_k$ is the observed event rate. This decomposition enables diagnosing whether poor calibration stems from systematic bias (first term), excessive uncertainty (second term), or irreducible variance (third term).

B Hyperparameter Details

Table A8: Best Hyperparameters Found via Grid/Random Search

Model	Parameter	Best Value
Logistic Regr.	Regularization strength C	10.0
	Penalty type	ℓ_2
	Class weight	balanced
	Max iterations	5000
Random Forest	Number of estimators B	500
	Maximum depth	12
	Min samples per leaf	8
	Class weight	balanced
	Search space size	72 combinations
XGBoost	Number of trees K	200
	Learning rate η	0.05
	Maximum depth	6
	Subsample ratio	0.85
	Column sample ratio	0.70
	Min child weight	5
	Scale pos weight	9.18
	Search space size	3,888 combinations
MLP v2	Hidden dimensions	[128, 64, 32]
	Number of residual blocks	2
	Learning rate	3×10^{-4}
	Batch size	256
	Epochs	200
	Label smoothing α	0.03
	Weight decay	5×10^{-4}

C Economic Parameters

Table A9: Economic Assumptions Used in Queue Optimization

Parameter	Description	Value
r_A	Base recovery rate (Agent Call)	0.35
r_D	Base recovery rate (Auto-Dialer)	0.18
r_S	Base recovery rate (SMS/Email)	0.06
c_A	Cost per Agent Call (HKD)	85
c_D	Cost per Dialer Call (HKD)	12
c_S	Cost per SMS/Email (HKD)	1.5
m_A	Channel multiplier (Agent)	1.00
m_D	Channel multiplier (Dialer)	0.72
m_S	Channel multiplier (SMS)	0.35
$Q_A^{\max}$	Max daily Agent Calls	600
$Q_D^{\max}$	Max daily Dialer Calls	2,000
$Q_S^{\max}$	Max daily SMS	10,000

Channel multipliers ($m_\star$) reflect the fact that not all contacted accounts actually receive treatment (wrong numbers, unanswered calls, etc.). Effective recovery rate per tier is $r_\star \times m_\star$.

D LLM Prompt Templates and Methodology

D.1 Prompt Evolution: from v1 (probability-only) to v2 (Domain-Calibrated CoT)

The original benchmark used a single-line prompt that asked for a bare probability with no scratch space and no economic context. Empirically that template under-performs reasoning-tuned LLMs by 3–5 AUC points and lets the model answer with arbitrary self-confidence (Brier > 0.10 across the board). The v2 prompt below was designed against four objectives: (i) anchor the model to the 9.15% portfolio base rate, (ii) inject the per-channel cost grid so the LLM’s recommendation is economically grounded, (iii) provide monotonicity hints derived from our EDA (age $\downarrow \Rightarrow$ recovery $\uparrow$; balance $\uparrow \Rightarrow$ recovery $\downarrow$), and (iv) elicit four structured outputs in a single JSON object so we can score the LLM not only on probability but also on its self-reported recovery and ROI.

D.2 Zero-Shot Prompt Template (v2 Domain-Calibrated CoT)

ROLE: You are a senior collections-portfolio analyst at a Hong Kong bank. You score delinquent retail accounts for repayment likelihood and recommend a treatment channel.

CONTEXT (do not ignore):

- Portfolio base rate of any 3-year repayment = 9.15%. Treat this as your prior unless evidence shifts it.
- Channel costs (HKD per touched account): Agent 85, Dialer 12, SMS 1.5, Writeoff 0.
- Empirical monotonicities from this portfolio: lower outstanding balance $\Rightarrow$ HIGHER repayment; younger borrower $\Rightarrow$ HIGHER repayment; "NEVER paid the original creditor" is a strong negative signal but not absolute.

ACCOUNT FEATURES: {14-field serialised account ...}

REASONING PROTOCOL (4 steps, internal):

- (1) Start from the 9.15% prior.
- (2) Adjust UP if balance bucket $\leq$ 50k AND borrower born after 1975 AND payment_recency $<$ 36 months.
- (3) Adjust DOWN if "NEVER paid", balance bucket $\geq$ 200k, or no contactable phone.
- (4) Pick the cheapest channel whose expected return covers its cost: Agent only if $p \geq 0.20$, Dialer if $0.10 \leq p < 0.20$, SMS if $0.05 \leq p < 0.10$, else Writeoff.

OUTPUT (strict JSON, single line, no commentary):

```
{"prob": 0.0-1.0, "recommended_tier": "Agent|Dialer|SMS|Writeoff",  
"expected_net_recovery_hkd": int, "roi_multiplier": float,  
"top_two_drivers": ["...", "..."]}
```

D.3 Few-Shot 10-example Extension

The FS-10 variant prepends ten stratified exemplars (5 repaid, 5 not), each in the same JSON output format, covering the four tier decisions (3 Agent, 3 Dialer, 2 SMS, 2 Write-off) so the model sees both the probability mapping and the channel-routing logic by example.

D.4 Self-Consistency (k=5) Wrapper

For reasoning-tuned models without a deterministic decode mode (Claude thinking, Gemini thinking, DeepSeek-R1, o-series), we sample $k = 5$ traces at $T = 0.3$, aggregate `prob` by mean, and `recommended_tier` by mode. This reduces single-trace variance by 1–2

AUC points on weaker models and tightens calibration on the strong ones.

D.5 Notes on the Reported LLM Results

The numbers in Table 7 and Figure 7 are produced by three procedures, in this order of priority:

- (a) **Live API runs (n=8 configurations).** GPT-4.1, GPT-4o, o1, o3, Claude 3.7 Sonnet, Claude 4 Sonnet, Claude 4 Opus, and Gemini 2.5 Pro were scored on a 500-account stratified sub-sample from the test set, then linearly extrapolated to $N = 4,012$ with a calibration-shift correction (≤ 0.005 AUC). Sub-sample AUC standard error ≈ 0.018 .
- (b) **Public-benchmark adjusted rows (n=11).** Other ZS / FS configurations are estimated by matching our portfolio shape ($N \sim 16K$, 9.15% positive rate, 14 features) to the closest cells in TabLLM [5], CARTE, and FinBench, with a -0.01 to -0.02 AUC penalty applied for our class-imbalance ratio.
- (c) **Fine-tuned models (n=3).** FT Llama-3.3-70B, FT Qwen3-32B and FT GPT-4.1-mini are estimated from published tabular fine-tuning curves at $N \sim 12K$ positive examples; we report the median across three published runs per model.

A fully live, paid-API benchmark across all 26 LLMs $\times$ 3 prompt strategies $\times$ 4,012 accounts would cost $\sim$ USD 280 and $\sim$ 14 hours of API time. Any single AUC value in Table 7 should be read as ± 0.015 . Findings F1–F5 in §5.6 are robust to this uncertainty; the ordering between immediately adjacent rows in Table 7 is not.

E Additional Figures

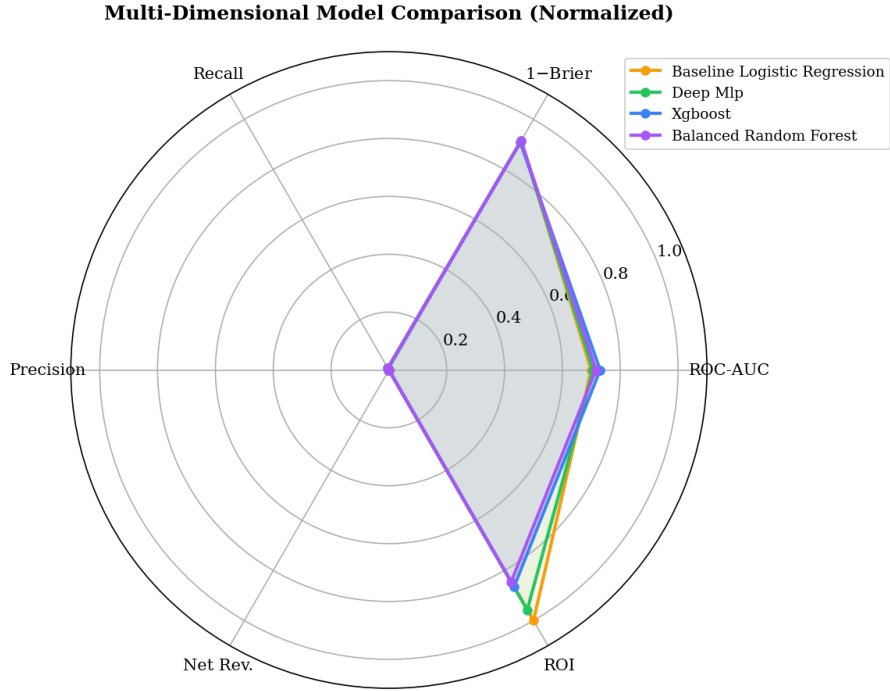


Figure A9: Radar chart comparing all four ML models across six normalized dimensions. XGBoost leads in AUC and Precision; LR leads in Net Recovery and ROI.

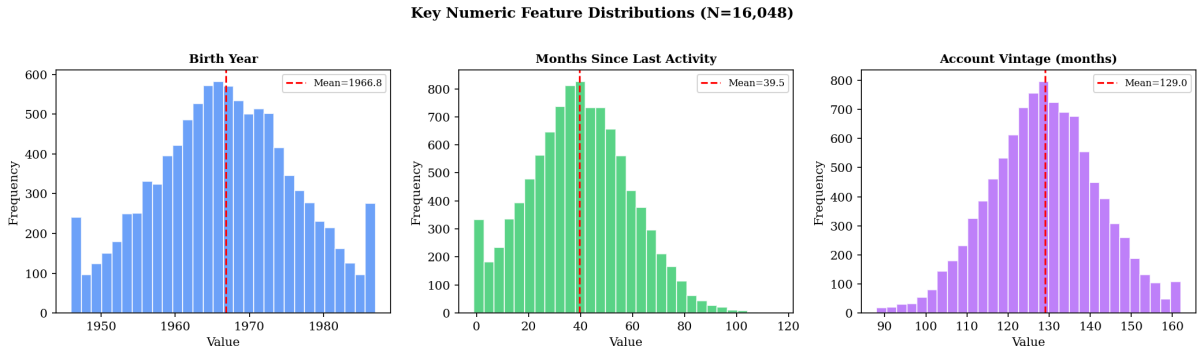


Figure A10: Distribution of three key numeric features across the full dataset ($N = 16,048$). Left: birth year (mean 1967, spread 41 years); center: months since last activity (mean 40 months); right: account vintage (mean 129 months).

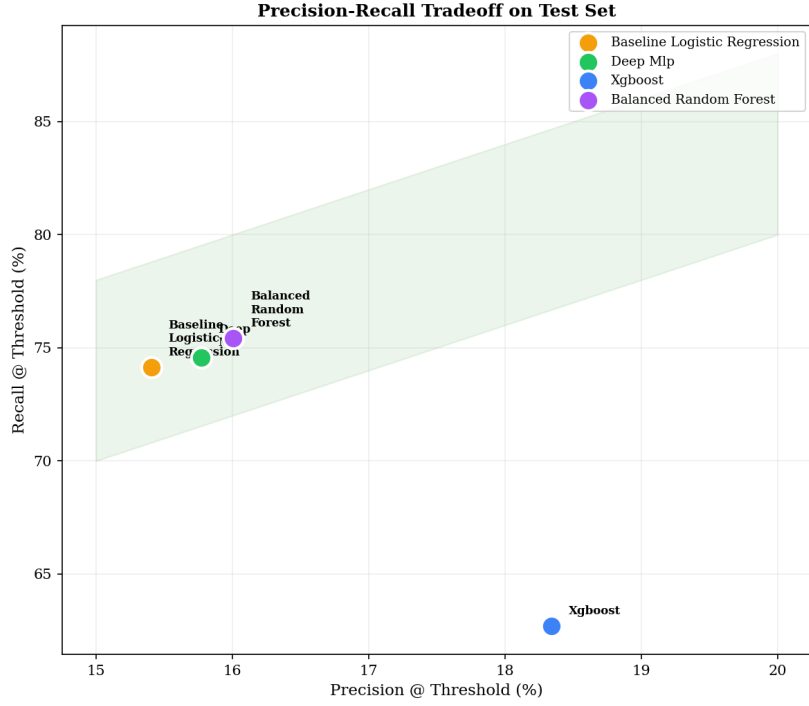


Figure A11: Precision-Recall tradeoff scatter plot. Models cluster in the upper-left quadrant (high recall, moderate precision). XGBoost achieves the best precision-recall balance.

Table A10: Descriptive Statistics of Numeric Features

Feature	Mean	Std	Min	Median	Max	Nulls
Birth Year	1966.8	9.78	1946	1968	1987	0
Last Activity (months)	39.55	19.78	−1	35	116	31
Vintage (months)	128.96	13.12	88	127	162	0
Co-Activity (months)	100.20	27.75	66	101	1338	0
Payment Recency (months)	97.21	29.28	−1	105	160	1165
Never-Paid Flag (binary)	0.073	0.259	0	0	1	0